94.3
Pass

mayuri Vilas Mestry

PDF generated at: 21 Feb 2026 08:52:40 UTC

View this report on HackerRank

Score

94.3% • 165.0 / 175
scored in eClerx - Java Coding Test - 3+ Years New in 67 min 47 sec on 21 Feb 2026 13:12:33 IST

Candidate Information

Email	mayurimestryofficial7@gmail.com
Test	eClerx - Java Coding Test - 3+ Years New
Candidate Packet	View
Taken on	21 Feb 2026 13:12:33 IST
Time taken	67 min 47 sec/ 90 min
Work Experience	> 5 years
Invited by	Vipul
Invited on	20 Feb 2026 13:12:59 IST

Skill Distribution

No.	Skill	Score
1	Spring Boot Advanced	50%

2	Spring Boot Intermediate	100%
3	Problem Solving Intermediate	100%
4	Java Intermediate	100%

Tags Distribution

Spring Boot	60%	Hard	50%
TDD	0%	Medium	100%
Graphs	100%	Greedy	100%
Sorting	100%	Java	100%

Questions

Spring Boot (Advanced) • 15 / 25

No.	Question	Skill	Score	Code Quality	Time Taken	Status
1	JSON Deserialization Multiple Choice	Spring Boot (Advanced)	5/5	-	1 min 16 sec	✓
2	Specification Multiple Choice	Spring Boot (Advanced)	5/5	-	44 sec	✓

3	Determine the Output Multiple Choice	Spring Boot (Advanced)	0/5	-	3 min 16 sec	✗
4	Response Multiple Choice	Spring Boot (Advanced)	0/5	-	1 min 25 sec	✗
5	Endpoint Security Multiple Choice	Spring Boot (Intermediate)	5/5	-	51 sec	✓

Problem Solving (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
6	Minimize Path Value Coding	Problem Solving (Intermediate)	75/75	-	45 min 50 sec	✓

Java (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
7	Java: DataVille's Optimal Storage Coding	Java (Intermediate)	75/75	-	13 min 25 sec	✓

1. JSON Deserialization

✓ Correct

Multiple Choice | Spring Boot | Hard

Question description

CheckerCreateRequest.java

```
// Annotation here
public class CheckerCreateRequest {
    private String name;
    private String surname;
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getSurname() {
        return surname;
    }
    public void setSurname(String surname) {
        this.surname = surname;
    }
}
```

CheckerController.java

```
@RestController
@RequestMapping("/checkers")
public class CheckerController {

    @Autowired
    CheckerService checkerService;

    @PostMapping
    public ResponseEntity createChecker(@RequestBody CheckerCreateRequest checkerCreateReque
st) {
        checkerService.createChecker(checkerCreateRequest);
        return new ResponseEntity(HttpStatus.CREATED);
    }
}
```

Request Json data

```
{
    "checker":{
        "name": "Checker",
```

```
        "surname":"Number2"
    }
}
```

A POST request with "Request Json data" is made to "/checkers". What annotation needs to be added to the "CheckerCreateRequest" class so that it can successfully deserialize and process the request?

Interviewer guidelines

@JsonRootName doesn't add checker as a wrapper attribute by itself. To make it successfully deserialize with given json annotations on D should be used together.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ @JsonRootName("checker")
- ○ @JsonName("checker")
- ○ @JsonTypeName("checker")
- ● **@JsonTypeInfo(include = JsonTypeInfo.As.WRAPPER_OBJECT, use = JsonTypeInfo.Id.NAME) @JsonTypeName("checker")** ✓

No comments.

2. Specification

Correct

Multiple Choice | Spring Boot | Hard

Question description

Which of the following options can be used for dynamic jpa queries with spring-data-jpa repositories?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ PreparedStatement
- ● Specification ✓
- ○ Criteria
- ○ Query

No comments.

3. Determine the Output

✗ Incorrect

Multiple Choice | Spring Boot | Hard

Question description

SpringQuestionApplication.java

```
package com.hackerrank.spring;
import org.springframework.boot.SpringApplication;
```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class SpringQuestionsApplication {
    public static void main(String[] args) {
        SpringApplication.run(SpringQuestionsApplication.class, args);
    }
}
```

Lie.java

```
package com.hackerrank;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;
@Component
public class Lie implements CommandLineRunner {
    @Override
    public void run(String... args) throws Exception {
        System.out.println("Truth is behind the lies");
        System.out.println("This is working perfectly");
        throw new Exception("Does this work?");
    }
}
```

Truth.java

```
package com.hackerrank.spring.reality;

import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

@Component
public class Truth implements CommandLineRunner {
    @Override
    public void run(String... args) throws Exception {
        System.out.println("Truth is in your eyes");
    }
}
```

What will be printed on the console when the "SpringBootApplication" class is executed?

Interviewer guidelines

Correct Answer:

Option [A] – "Truth is in your eyes"
Explanation: Spring Boot scans only the package of the `@SpringBootApplication` class and its subpackages. `Truth` is under a subpackage and runs. `Lie` is in a parent package, so it is **not scanned** and its runner does not execute.

Incorrect Answers:

Option [B] – Multiple prints + exceptions
Explanation: `Lie` never runs, so its prints and exception do not occur.
Option [C] – Prints from Lie only
Explanation: `Truth` is discovered and executed; `Lie` is not.
Option [D] – Prints from Lie, then Truth
Explanation: `Lie` is outside the scan path and never executed.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- Truth is in your eyes ✓
- Truth is in your eyes Truth is behind the lies This is working perfectly And exception occurs and application stops
- Truth is behind the lies This is working perfectly And exception occurs and application stops
- **Truth is behind the lies This is working perfectly And exception occurs containing "Does this work?" Truth is in your eyes**

No comments.

4. Response

Incorrect

Multiple Choice | Hard | Spring Boot | TDD

Question description

Which response will cause the following test case to fail?

```
@Test
   @DisplayName("Filter Price Check 3")
   public void FilterPriceCheck3() throws Exception {
      mockMvc.perform(get("/filter/price/150/500"))
            .andDo(print())
            .andExpect(jsonPath("$[*].product", containsInAnyOrder("Product 1", "Product 2" , "Produc
t 5")))
   }
```

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [ ] [{"product" : "Product 5"} , {"product" : "Product 1"}, {"product" : "Product 2"}]
- [ ] [{"product" : "Product 5"} , {"product" : "Product 2"}, {"product" : "Product 1"}]
- [x] **[{"product" : "Product 1"} , {"product" : "Product 2"}, {"product" : "Product 5"}]**

[{"product" : "product 5"} , {"product" : "product 2"}, {"product" : "product 1"}]

No comments.

5. Endpoint Security

Correct

Multiple Choice | Spring Boot | Medium

Question description

In Spring Boot Security, the option to enable secure endpoints with @PreAuthorize annotation is:

Interviewer guidelines

@EnableGlobalMethodSecurity by default doesn't enable this feature. To enable this prePostEnabled=true value must be set

Candidate's Solution

Options: (Expected answer indicated with a tick)

- Configuration class annotated with @EnableGlobalMethodSecurity
- Configuration class annotated with @EnableGlobalMethodSecurity(securedEnabled = true)

- [x] Configuration class annotated with @EnableGlobalMethodSecurity(prePostEnabled = true)
- [ ] None above

No comments.

6. Minimize Path Value

Correct

Coding Graphs Medium

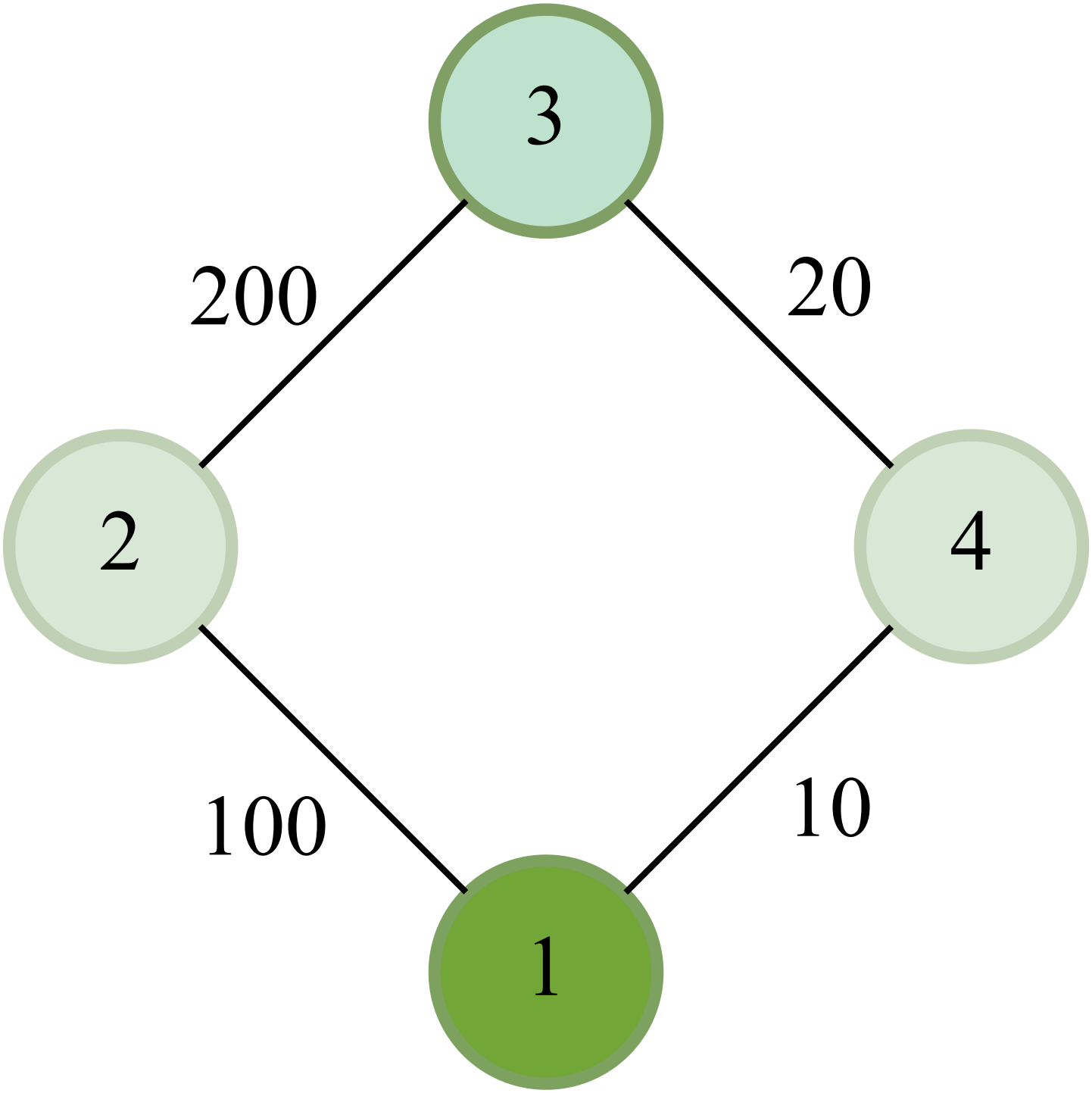
Question description

You have a weighted undirected graph with *graph_nodes* nodes and *m* edges. The stress level of a path between two nodes is defined as the weight of the heaviest edge in that path.

Given a source node *source* and a destination node *destination*, find the minimum possible stress level of a path. If no such path exists, return -1.

Example

Consider the following example below, where *source=1* and *destination=3.*



There are two paths, from node 1 to node 3.

1. The max weighted edge/stress level in path 1 -> 2 -> 3 is 200.
2. The max weighted edge/stress level in the path 1 -> 4 -> 3 is 20.

Return 20, the lower stress level from the second path.

Function Description

Complete the function *getMinimumStress* in the editor with the following parameter(s):

int graph_nodes: number of nodes
int graph_from[m]: one end node for an edge
int graph_to[m]: one end node for an edge
int graph_weight[m]: the weights of the edges
int source: the source node
int destination: the destination node

Returns

int: the value of the least stressful path, or -1 if no path exists

Note: Even though edge endpoints are called *graph_from[i]* and *graph_to[i]*, this is an undirected graph.

Constraints

- $1 \le graph_nodes \le 10^5$
- $1 \le |graph_from| = |graph_to| = |graph_weight| \le 10^5$ *(Where |A| denotes the size of array A)*
- $1 \le graph_from[i], graph_to[i], source, destination \le graph_nodes$
- $0 \le graph_weight[i] \le 10^9$

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains two space-separated integers *graph_nodes* and *m,* the number of nodes, and the number of edges of the graph.
Each of the following *m* lines contains three space-separated integers *u, v,* and *w,* meaning there is an edge connecting the nodes *u* and *v* of weight *w.*
The next line contains a single integer, denoting the index of the "*source*" node.
The next line contains a single integer, denoting the index of the "*destination*" node.

▼ SAMPLE CASE 0

Sample Input For Custom Testing

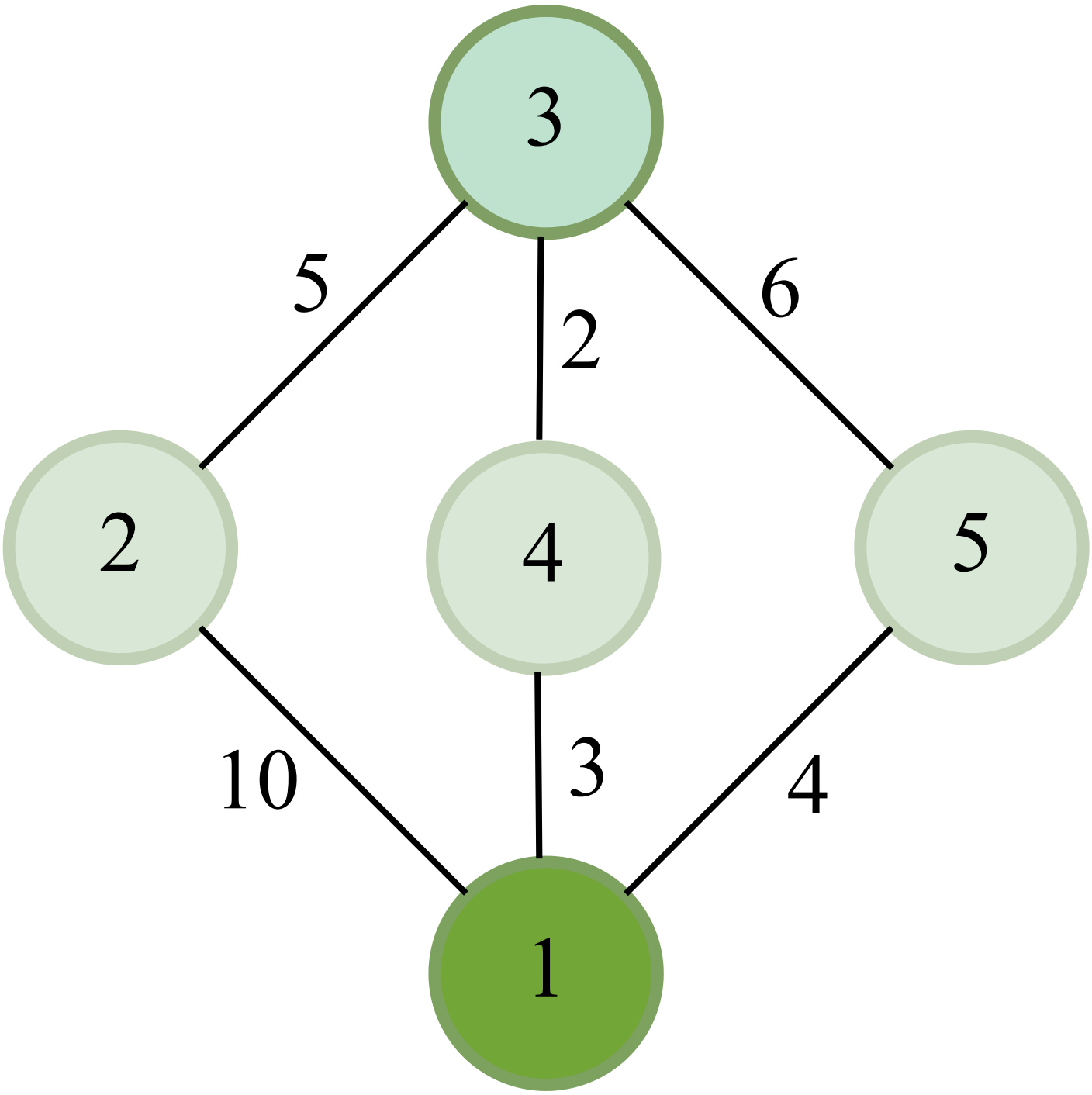
```
STDIN       FUNCTION
-----       --------
5 6         graph_nodes = 5, m = 6
1 2 10      u = 1, v = 2, w = 10
```

```
2 3 5
1 4 3
4 3 2
1 5 4
5 3 6
1          source = 1
3          destination = 3
```

Sample Output

```
3
```

Explanation



There are three paths, from node 1 to node 3 :

1. 1 -> 2 -> 3. Edges have weights 10 and 5, the maximum weighted edge, and the stress level is 10.
2. 1 -> 4 -> 3. Edges have weights 3 and 2, and the stress level = 3.
3. 1 -> 5 -> 3. Edges have weights 4 and 6, and the stress level = 6.

▼ SAMPLE CASE 1

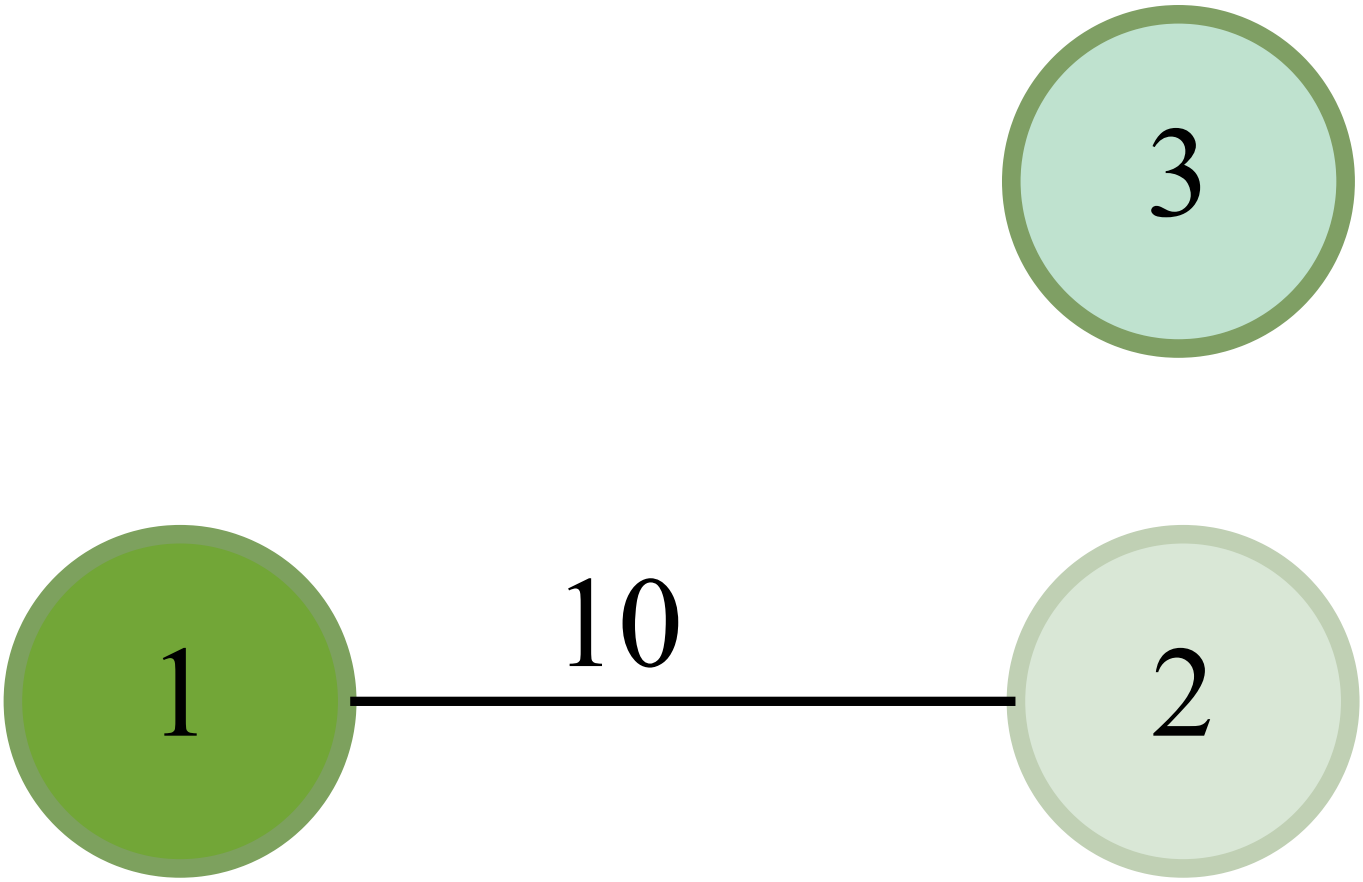
Sample Input For Custom Testing

```
STDIN       FUNCTION
-----       --------
3 1         graph_nodes = 3, m = 1
1 2 10      a = 1, b = 2, w = 10
1           source = 1
3           destination = 3
```

Sample Output

```
-1
```

Explanation



There is no path, from node 1 to node 3.

Interviewer guidelines

▼ SOLUTION

Python Solution

```
sys.setrecursionlimit(10**5)


def getMinimumStress(graph_nodes, graph_from, graph_to, graph_weight, source, destination):

    adj = [[] for i in range(graph_nodes + 1)]
    vis = [False] * (graph_nodes + 1)

    for x, y, w in zip(graph_from, graph_to, graph_weight):
        adj[x].append((y, w))
        adj[y].append((x, w))

    def dfs(node, mx):
        vis[node] = True
        if node == destination:
            return True
        ans = False
        for i, w in adj[node]:
            if w <= mx and not vis[i]:
                ans |= dfs(i, mx)
        return ans

    if not dfs(source, max(graph_weight)):
        return -1

    lo, hi = 0, max(graph_weight)
    while hi - lo > 1:
        vis = [False] * (graph_nodes + 1)
        mid = (hi + lo) // 2
        if dfs(source, mid):
            hi = mid
        else:
            lo = mid
    return hi
```

Candidate's Solution

Language used: **Java 17**

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
```

```
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;


import java.io.*;

import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;



class Result {


static class Edge{

    int to;
    int weight;

    Edge(int to, int weight)
    {
        this.to=to;
        this.weight=weight;
    }
}



    /*
     * Complete the 'getMinimumStress' function below.
     *
     * The function is expected to return an INTEGER.
     * The function accepts following parameters:
     *  1. WEIGHTED_INTEGER_GRAPH graph
     *  2. INTEGER source
```

```
     *  3. INTEGER destination
     */


    /*
     * For the weighted graph, <name>:
     *
     * 1. The number of nodes is <name>Nodes.
     * 2. The number of edges is <name>Edges.
     * 3. An edge exists between <name>From[i] and <name>To[i]. The weight of the edge is <name>Weight[i].
     *
     */



    public static int getMinimumStress(int graphNodes, List<Integer> graphFrom, List<Integer> graphTo, List<Integer> graphWeight, int source, int destination) {
      List<List<Edge>> adj = new ArrayList<>();
        for (int i = 0; i <= graphNodes; i++){
            adj.add(new ArrayList<>());
        }

        for(int i = 0; i< graphFrom.size(); i++)
        {
            int u = graphFrom.get(i);
            int v = graphTo.get(i);
            int w = graphWeight.get(i);

            adj.get(u).add(new Edge(v, w));
            adj.get(v).add(new Edge(u, w));
        }

        int[] dist = new int [graphNodes+1];
        Arrays.fill(dist,Integer.MAX_VALUE);
        dist[source]=0;

        PriorityQueue<int[]> pq= new PriorityQueue<>(Comparator.comparingInt(a->a[1]));
        pq.offer(new int []{source,0});

        while(!pq.isEmpty()){
            int [] cur= pq.poll();
            int node = cur[0];
            int cost =  cur[1];
```

```java

            if (node==destination) return cost;
            if (cost> dist[node]) continue;

            for(Edge edge : adj.get(node)){
                int nextCost =Math.max(cost, edge.weight);

                if (nextCost<dist[edge.to]){
                    dist[edge.to] = nextCost;
                    pq.offer(new int[]{edge.to,nextCost});
                }
            }
        }
        return -1;
    }

}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        String[] graphNodesEdges = bufferedReader.readLine().replaceAll("\\s+$", "").split(" ");

        int graphNodes = Integer.parseInt(graphNodesEdges[0]);
        int graphEdges = Integer.parseInt(graphNodesEdges[1]);

        List<Integer> graphFrom = new ArrayList<>();
        List<Integer> graphTo = new ArrayList<>();
        List<Integer> graphWeight = new ArrayList<>();

        IntStream.range(0, graphEdges).forEach(i -> {
            try {
                String[] graphFromToWeight = bufferedReader.readLine().replaceAll("\\s+$", "").split(" ");

                graphFrom.add(Integer.parseInt(graphFromToWeight[0]));
                graphTo.add(Integer.parseInt(graphFromToWeight[1]));
                graphWeight.add(Integer.parseInt(graphFromToWeight[2]));
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
```

```
        });

        int source = Integer.parseInt(bufferedReader.readLine().trim());

        int destination = Integer.parseInt(bufferedReader.readLine().trim());

        int result = Result.getMinimumStress(graphNodes, graphFrom, graphTo, graphWeight, source, destination);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        String[] graphNodesEdges = bufferedReader.readLine().replaceAll("\\s+$", "").split(" ");

        int graphNodes = Integer.parseInt(graphNodesEdges[0]);
        int graphEdges = Integer.parseInt(graphNodesEdges[1]);

        List<Integer> graphFrom = new ArrayList<>();
        List<Integer> graphTo = new ArrayList<>();
        List<Integer> graphWeight = new ArrayList<>();

        IntStream.range(0, graphEdges).forEach(i -> {
            try {
                String[] graphFromToWeight = bufferedReader.readLine().replaceAll("\\s+$", "").split(" ");

                graphFrom.add(Integer.parseInt(graphFromToWeight[0]));
                graphTo.add(Integer.parseInt(graphFromToWeight[1]));
                graphWeight.add(Integer.parseInt(graphFromToWeight[2]));
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
```

```
        });

        int source = Integer.parseInt(bufferedReader.readLine().trim());

        int destination = Integer.parseInt(bufferedReader.readLine().trim());

        int result = Result.getMinimumStress(graphNodes, graphFrom, graphTo, graphWeight, source, destination);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
TestCase 0	Easy	Sample	Success	1	0.0587 sec	33.8 KB
TestCase 1	Easy	Sample	Success	1	0.0466 sec	33.5 KB
TestCase 2	Easy	Sample	Success	1	0.0564 sec	33.7 KB
TestCase 3	Easy	Hidden	Success	6	0.541 sec	100 KB
TestCase 4	Easy	Hidden	Success	6	0.5106 sec	87.3 KB
TestCase 5	Easy	Hidden	Success	6	0.4828 sec	103 KB
TestCase 6	Easy	Hidden	Success	6	0.5462 sec	115 KB

TestCase 7	Easy	Hidden	Success	6	0.6085 sec	102 KB
TestCase 8	Easy	Hidden	Success	6	0.4888 sec	94.9 KB
TestCase 9	Easy	Hidden	Success	6	0.4703 sec	106 KB
TestCase 10	Easy	Hidden	Success	6	0.6617 sec	108 KB
TestCase 11	Easy	Hidden	Success	6	0.657 sec	101 KB
TestCase 12	Easy	Hidden	Success	6	0.3214 sec	66.8 KB
TestCase 13	Easy	Hidden	Success	6	0.4608 sec	71 KB
TestCase 14	Easy	Hidden	Success	6	0.71 sec	101 KB

No comments.

7. Java: DataVille's Optimal Storage

Correct

Coding Greedy Sorting Medium Java

Question description

In the digital city of DataVille, efficient data management is critical. The city's database is represented as a binary array, *databaseStorage*, containing *n* bits of data. Each bit represents one of two types of data:

- Standard data: Represented by 0.
- Priority data: Represented by 1.

To optimize efficiency during peak operational hours, the number of transitions between 0s and 1s in the array must be minimized. Transitions are points in the array where adjacent elements differ (e.g., a 0 followed by a 1 or a 1 followed by a 0).

Operations:

- You can swap any two bits in the array as many times as you like.

The goal is to rearrange the bits to minimize transitions with the minimum number of swaps.

Example

Consider that *databaseStorage* = [1, 0, 1, 0, 1]. One optimal strategy is to swap *databaseStorage[0]* and *databaseStorage[3]*. After performing this operation, the array is [**0**, 0, 1, **1**, 1], which requires only one transition between *databaseStorage[1]* and *databaseStorage[2]*. This requires only 1 swap, so the return value is 1.

Function Description

Complete the function g*etOptimalStorage* in the editor below.

Function Parameters

int databaseStorage[n]: the current organization of the database's storage, with 0s for standard data and 1s for priority data

Returns

int: the minimum number of operations required to minimize transitions between 0s and 1s

Constraints

- $2 \le n \le 10^5$
- *databaseStorage[i]* $\in \{0, 1\}$

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer, *n,* the size of *databaseStorage*.
Each of the next lines contains an integer, *databaseStorage[i].*

▼ SAMPLE CASE 0

Sample Input 0

```
STDIN       FUNCTION
-----       --------
7           databaseStorage[] size n = 7
1           databaseStorage = [1, 0, 1, 1, 0, 0, 1]
0
1
1
0
0
1
```

Sample Output 0

```
1
```

Explanation

One optimal strategy is to swap *databaseStorage[1]* (which is 0) with *databaseStorage[6]* (which is 1). After performing this operation, the array becomes [1, **1**, 1, 1, 0, 0, **0**]. This solution requires only 1 swap, so the output is 1.

▼ SAMPLE CASE 1

Sample Input 1

```
STDIN     FUNCTION
-----     --------
9         databaseStorage[] size n = 9
1         databaseStorage = [1, 0, 0, 1, 1, 0, 0, 0, 1]
0
0
1
1
0
0
0
1
```

Sample Output 1

```
2
```

Explanation

One optimal strategy of operations is:

1. Swap *databaseStorage*[1] and *databaseStorage*[8]. After performing this operation, the array becomes [1, **1**, 0, 1, 1, 0, 0, 0, **0**].
2. Swap *databaseStorage*[2] and *databaseStorage*[4]. After performing this operation, the array becomes [1, 1, **1**, 1, **0**, 0, 0, 0, 0].

Now there is only one transition. The minimum number of swaps is 2.

Interviewer guidelines

▼ SOLUTION

The solution is to count the number of 0s and 1s in the array. Then, we count the number of 0s and 1s that are out of place. The answer is the minimum of the two counts.

```
class Result {

    /*
     * Complete the 'getOptimalStorage' function below.
     *
     * The function is expected to return an INTEGER.
     * The function accepts INTEGER_ARRAY databaseStorage as parameter.
     */

    public static int getOptimalStorage(List<Integer> databaseStorage) {
        int zeros = 0, ones = 0;

        for (int brick : databaseStorage) {
            if (brick == 0) {
                zeros++;
            } else {
                ones++;
            }
        }

        int startZero = 0, startOne = 0;

        for (int i = 0; i < ones; ++i) {
            if (databaseStorage.get(i) != 1) {
                startOne++;
            }
        }
```

```
        for (int i = 0; i < zeros; ++i) {
            if (databaseStorage.get(i) != 0) {
                startZero++;
            }
        }

        return Math.min(startZero, startOne);
    }
}
```

▼ COMPLEXITY

Time Complexity - O(n)

To traverse over the array, we use a loop that runs n times. The complexity is O(n).

Space Complexity - O(1)

We use only 2 variables, hence the space complexity is O(1).

Candidate's Solution

Language used: **Java 17**

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;


class Result {



    /*
     * Complete the 'getOptimalStorage' function below.
     *
```

```
     * The function is expected to return an INTEGER.
     * The function accepts INTEGER_ARRAY databaseStorage as parameter.
     */

    public static int getOptimalStorage(List<Integer> databaseStorage) {

        int n = databaseStorage.size();

        int count0 = 0;
        for(int val : databaseStorage)
        {
            if(val == 0)
            {
                count0++;
            }
        }
        int count1 = n - count0;

        int misplacedForZeroFirst = 0;
        for (int i =0; i < count0; i++ )
        {
            if(databaseStorage.get(i)==1){
                  misplacedForZeroFirst++;
            }

        }

         int misplacedForOneFirst = 0;
        for (int i = 0; i < count1; i++ )
        {
            if(databaseStorage.get(i)==0){
            misplacedForOneFirst++;
            }

        }
        return Math.min(misplacedForZeroFirst, misplacedForOneFirst);

    }

}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));
```

```
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        int databaseStorageCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> databaseStorage = IntStream.range(0, databaseStorageCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int result = Result.getOptimalStorage(databaseStorage);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
TestCase 0	Easy	Sample	Success	1	0.0491 sec	33.8 KB
TestCase 1	Easy	Sample	Success	1	0.0527 sec	33.7 KB
TestCase 2	Easy	Sample	Success	1	0.046 sec	33.7 KB
TestCase 3	Easy	Hidden	Success	3	0.0495 sec	33.6 KB

TestCase 4	Easy	Hidden	Success	3	0.0473 sec	33.6 KB
TestCase 5	Easy	Hidden	Success	3	0.0459 sec	33.9 KB
TestCase 6	Easy	Hidden	Success	4	0.0489 sec	33.9 KB
TestCase 7	Easy	Hidden	Success	4	0.0457 sec	33.7 KB
TestCase 8	Easy	Hidden	Success	5	0.0529 sec	33.8 KB
TestCase 9	Easy	Hidden	Success	5	0.0591 sec	33.6 KB
TestCase 10	Easy	Hidden	Success	5	0.0529 sec	33.8 KB
TestCase 11	Medium	Hidden	Success	7	0.0702 sec	36.4 KB
TestCase 12	Medium	Hidden	Success	8	0.0815 sec	36.2 KB
TestCase 13	Hard	Hidden	Success	10	0.0732 sec	36 KB
TestCase 14	Hard	Hidden	Success	15	0.1814 sec	62.2 KB

No comments.